

Herald



Herald — commons_auth Module Guide (Operator / Developer)

Field	Value
Revision	1
Created	2026-05-31
Last modified	2026-05-31
Status	active
Status summary	Nano-detail per-module reference for commons_auth (Wave 3a) — the JWT verifier (HMAC HS256 + JWKS RS256/ES256 hybrid) and Gin middleware factory that JWT-gates every /v1/* route on Herald's serving flavors (cherald, sherald, iherald). Documents the Verifier interface, the two verify modes (ModeHMAC / ModeJWKS), NewVerifier / NewVerifierFromEnv, the Redis-cached JWKS keys with in-memory fast-path + in-memory-only fallback, GinMiddleware, the ContextKeyClaims Gin key, the TenantFromClaims / SubjectFromClaims helpers, every env var the module reads, and 401 troubleshooting. ANTI-BLUFF: every section documents only what the source under commons_auth/ actually does as of this revision.
Issues	(none specific to this guide)
Continuation	bump when the module gains an audience (aud) / issuer (iss) check beyond the presence-only requireClaims, when a non-Gin middleware adapter lands, or when the JWKS key-rotation re-fetch grows a backoff/rate-limit.

Table of contents

- [§1. Overview](#)
- [§2. The API](#)
- [§3. The two verify modes](#)
- [§4. JWKS key caching: Redis + in-memory fallback](#)
- [§5. The Gin middleware and how flavors wire it](#)
- [§6. Claims helpers and the context key](#)
- [§7. Configuring JWT auth \(operator\)](#)
- [§8. Troubleshooting 401s](#)
- [§9. Testing notes](#)
- [§10. References](#)

§1. Overview

`commons_auth` (Go package `commons_auth`, module path `github.com/vasic-digital/herald/commons_auth`) provides **JWT verification** and a **Gin middleware factory** for Herald's serving flavor binaries. Per the package doc at the top of `verifier.go` (Wave 3 design §4), every serving flavor's `main.go` threads `GinMiddleware` into `cli.ServeOpts.Middleware` so it gates every route.

The module exposes one verification abstraction — the `Verifier` interface — with two concrete implementations selected by a `Mode`:

1. `ModeHMAC` (`hmacVerifier`) — symmetric HS256 with a pre-shared secret.
2. `ModeJWKS` (`jwksVerifier`) — asymmetric RS256/ES256 family, with public keys fetched from a JWKS URL, cached in Redis when available, and held in an in-memory map as the hot path.

The module is **Herald-internal-by-design**. Its package doc records the §11.4.74 catalogue-check verdict: no existing `vasic-digital/HelixDevelopment` module satisfied this shape, so it was vendored here. See `docs/catalogue-checks/HRD-099-commons-auth.md` (the source comments still cite the executing-time anchor `HRD-093`; `HRD-099` is the canonical anchor after the post-Wave-3a renumber).

Third-party dependencies (per `commons_auth/go.mod`): `github.com/golang-jwt/jwt/v5 v5.2.1` (token parse + verify), `github.com/redis/go-redis/v9 v9.6.1` (JWKS key cache), `github.com/gin-gonic/gin v1.12.0` (middleware), `github.com/google/uuid v1.6.0` (tenant-claim parsing), `github.com/jonboulle/clockwork v0.4.0` (injectable clock for expiry tests). RSA/EC JWK parsing uses only the Go standard library (`crypto/rsa`, `crypto/ecdsa`, `crypto/elliptic`, `encoding/base64`, `math/big`).

§2. The API

§2.1 Verifier

```
type Verifier interface {  
    Verify(token string) (map[string]any, error)  
}
```

A single method. `Verify` parses and cryptographically validates the JWT, enforces the required-claims presence check, and returns the claims as a `map[string]any` (the `jwt.MapClaims` payload), or an error. The concrete implementations are unexported (`hmacVerifier`, `jwksVerifier`) — callers always hold the interface.

§2.2 Mode

```
type Mode string  
  
const (  
    ModeHMAC Mode = "hmac" // pre-shared secret, HS256
```

```

    ModeJWKS Mode = "jwks" // JWKS-fetched public keys, RS256/
        ES256 family
)

```

§2.3 Config

```

type Config struct {
    Mode          Mode
    HMACSecret    []byte
    JWKSURL       string
    JWKSCacheTTL  time.Duration
    RequiredClaims []string
    Clock         clockwork.Clock
}

```

Field	Used in	Meaning
Mode	both	Selects which implementation <code>NewVerifier</code> returns.
HMACSecret	HMAC	Raw secret bytes for HS256. Empty → <code>NewVerifier</code> errors.
JWKSURL	JWKS	HTTPS JWKS endpoint. Empty → <code>NewVerifier</code> errors.
JWKSCacheTTL	JWKS	Redis cache TTL for fetched keys. Zero → defaults to 5 minutes .
RequiredClaims	both	Claims that MUST be present (and, when string-typed, non-empty). Empty → defaults to <code>["tenant", "sub"]</code> (<code>defaultRequiredClaims</code>).
Clock	both	Injectable clock threaded into token-expiry validation via <code>jwt.WithTimeFunc</code> . Nil → <code>clockwork.NewRealClock()</code> .

§2.4 Constructors

Function	Signature	Behaviour
<code>NewVerifier</code>	<code>NewVerifier(cfg Config, rdb redis.Cmdable) (Verifier, error)</code>	Applies the clock + required-claims defaults, then switches on <code>cfg.Mode</code> . HMAC mode requires a non-empty <code>HMACSecret</code> (else error). JWKS mode requires a non-empty <code>JWKSURL</code> (else error) and defaults <code>JWKSCacheTTL</code> to 5m. An unknown mode returns <code>unknown mode %q</code> . <code>rdb</code> is used only in JWKS mode (HMAC ignores it); <code>rdb == nil</code> in JWKS mode is allowed — the verifier falls back to in-memory-only caching (§4).
<code>NewVerifierFromEnv</code>	<code>NewVerifierFromEnv(rdb redis.Cmdable) (Verifier, error)</code>	Reads <code>HERALD_AUTH_MODE</code> + the mode-specific env vars (§7), constructs a <code>Config</code> , and delegates to <code>NewVerifier</code> . An

Function	Signature	Behaviour
		unset <code>HERALD_AUTH_MODE</code> is an error. This is the constructor the flavor <code>serve</code> commands call.

Why build the verifier at serve-time, not in `main()`. The flavor `serve` `RunE` builds the verifier (and Redis) inside the command, not at process init, so non-serving CLI subcommands (`version`, `doctor`, ...) never require auth env to be set. See the comment in `cherald/cmd/cherald/main.go` around the `NewVerifierFromEnv` call.

§3. The two verify modes

Both modes share the same post-parse path: `assert parsed.Valid`, `cast parsed.Claims` to `jwt.MapClaims`, run `requireClaims`, and return the map. They differ only in the `jwt.Parse` key function.

§3.1 HMAC (HS256) — `hmac.go`

`hmacVerifier.Verify` calls `jwt.Parse` with a key function that **rejects any algorithm other than HS256** (`t.Method.Alg() != jwt.SigningMethodHS256.Alg() → error`) and returns `cfg.HMACSecret` as the key. Expiry is validated against `cfg.Clock.Now` via `jwt.WithTimeFunc`. This algorithm pin is the standard defence against the JWT alg-confusion attack (a token forged with `alg: none` or an asymmetric alg is refused).

§3.2 JWKS (RS256/ES256 family) — `jwtks.go`

`jwtksVerifier.Verify` calls `jwt.Parse` with `keyFunc`, which:

1. **Restricts the algorithm** to the explicitly supported set — RS256/RS384/RS512 and ES256/ES384/ES512. Anything else (including `alg: none` and HS256) is rejected with `unexpected signing method`.
2. Reads the `kid` header (empty `kid` → `token missing kid`).
3. Looks the `kid` up in the in-memory key map under an `RLock`; on a hit returns the key immediately.
4. On a miss, calls `refresh(ctx, force=false)` (Redis-then-HTTPS, §4) and retries the lookup; a still-missing `kid` yields the sentinel `errKidNotFound`.

`Verify` wraps `keyFunc` with a **key-rotation race handler**: if the first `jwt.Parse` fails with `errKidNotFound`, it forces one re-fetch (`refresh(ctx, force=true)`), bypassing the Redis cache and pulling fresh JWKS over HTTPS) and re-parses once. If that re-fetch fails the original error is returned with the refresh error appended.

Supported JWK key types (`parseJWKSKeys`, per RFC 7517/7518):

- `key=RSA` with `base64url n+e → *rsa.PublicKey` (exponent validated `>= 1`).
- `key=EC` with `crv+x+y, crv ∈ {P-256, P-384, P-521} → *ecdsa.PublicKey` (the point is checked on-curve via `curve.IsOnCurve`).
- Other `key` values (`oct symmetric`, `OKP/Ed25519`) are **skipped silently** so a JWKS document carrying extra key types still loads — explicit per RFC 7517 §6. If **no** supported keys survive parsing, the load errors (JWKS document contained no supported keys). Every JWK must carry a non-empty `kid` (the lookup index) or the load errors.

§3.3 Required-claims enforcement (both modes)

`requireClaims(claims, required)` asserts every key in `required` is present; when a present value is a **string** it must be non-empty (token claim %q is empty). Non-string types (e.g. a numeric exp) pass the presence check without a value assertion. This is a **presence/non-emptiness** check only — there is no audience (`aud`), issuer (`iss`), or value-equality check in the current source. The default required set is `["tenant", "sub"]`.

§4. JWKS key caching: Redis + in-memory fallback

The JWKS verifier holds keys in an in-memory map (`inMemoryKeys: kid → *rsa.PublicKey | *ecdsa.PublicKey`) guarded by a `sync.RWMutex`. That map is the hot path — every `Verify` reads it under an `RLock` first.

`refresh(ctx, force)` is the fetch path:

1. **Redis read (unless force).** When `rdb != nil` and not forcing, it reads the cached JWKS bytes from the Redis key `herald:auth:jwks:<sha256(JWKSURL)>` (`cacheKey`, computed once in `newJWKSVerifier`). A non-empty hit is parsed straight into the in-memory map and `refresh` returns.
2. **HTTPS fetch.** On a Redis miss, a Redis error, or `force=true`, it does an HTTP GET of `cfg.JWKSURL` (`http.DefaultClient`). A non-200 status is an error (JWKS HTTP %d).
3. **Redis write-back.** When `rdb != nil`, the fetched body is written back to Redis with TTL `cfg.JWKSCacheTTL`. **Redis errors here are non-fatal** — they are swallowed (`_ = ...`) because the keys are still loaded into memory.
4. **In-memory populate.** `loadJWKS` parses the body and atomically swaps `inMemoryKeys` (under the write lock) plus records `fetchedAt`.

In-memory-only fallback. When `rdb == nil` (Redis unavailable / not wired), `refresh` skips the Redis read and Redis write-back entirely and goes straight to HTTPS → in-memory. So **JWKS verification works without Redis** — Redis is a cross-process/cross-restart cache optimisation, not a hard dependency. HMAC mode never touches Redis at all.

§5. The Gin middleware and how flavors wire it

§5.1 GinMiddleware — `middleware.go`

```
func GinMiddleware(v Verifier) gin.HandlerFunc
```

The returned handler, per request:

1. Reads the `Authorization` header. Missing header, or a header without the Bearer prefix, or an empty token after the prefix → `AbortWithStatusJSON(401, {"error": "missing bearer token"})`.
2. Calls `v.Verify(token)`. Any verify error → `AbortWithStatusJSON(401, {"error": "invalid token"})`. The body schema is intentionally minimal — clients should not depend on internal error formatting beyond the top-level "error" field.
3. On success, stores the claims map in the Gin context under `ContextKeyClaims` and calls `c.Next()`.

§5.2 How serving flavors thread it

The serving flavors — `cherald`, `sherald`, `iherald` — wire it identically in their `serve RunE`:

```
// (cherald/sherald/iherald serve RunE — verbatim shape)
rdb, err := buildRedis()                                // Redis
    optional
if err != nil { return err }
verifier, err := commons_auth.NewVerifierFromEnv(rdb)
if err != nil { return fmt.Errorf("build verifier: %w", err) }

opts := cli.ServeOpts{
    Branding:    branding,
    Routes:      flavhttp.Routes(...),
    Middleware:
        []gin.HandlerFunc{commons_auth.GinMiddleware(verifier)},
}
return cli.RunServe(ctx, opts, port)
```

`cli.ServeOpts.Middleware` is `[]gin.HandlerFunc`. `RunServe` registers the health/metrics routes and its auto-wired middleware (`Alt-Svc`, `Brotli`, `TOON`), then loops over `opts.Middleware` calling `r.Use(mw)` (see `commons/cli/serve.go` — for `_`, `mw := range opts.Middleware { r.Use(mw) }`). Because every `/v1/*` route is registered after that `r.Use`, the JWT gate applies to all of them. The non-serving flavor `pherald` does not auth-gate this way (it is the inbound runtime, not a `/v1` server).

§6. Claims helpers and the context key

§6.1 ContextKeyClaims

```
const ContextKeyClaims = "herald.auth.claims"
```

The Gin context key under which the authenticated claims map is stored. Downstream handlers retrieve it with `c.Get(ContextKeyClaims)` (returns `any, bool`; type-assert to `map[string]any`).

§6.2 TenantFromClaims and SubjectFromClaims — `claims.go`

```
func TenantFromClaims(claims map[string]any) (uuid.UUID, error)
func SubjectFromClaims(claims map[string]any) (string, error)
```

- `TenantFromClaims` reads the "tenant" claim, requires it to be a string, and parses it as a UUID. Missing claim → missing 'tenant' claim; wrong type → 'tenant' claim must be a string, got %T; unparseable → the `uuid.Parse` error. Returns `uuid.Nil` on any failure.
- `SubjectFromClaims` reads the "sub" claim and requires it to be a string. Missing → missing 'sub' claim; wrong type → 'sub' claim must be a string, got %T.

These are the canonical accessors for the two default-required claims; handlers use them after pulling the map out of the Gin context.

§7. Configuring JWT auth (operator)

`NewVerifierFromEnv` reads these environment variables (resolution order is the project-wide rule: exported shell vars from `.bashrc/.zshrc` first, `.env` fallback — see `docs/guides/OPERATOR_CREDENTIALS.md`):

Env var	Mode Required		Meaning
HERALD_AUTH_MODE	both	yes	"hmac" or "jwks". Unset → the flavor <code>serve</code> fails to start with <code>HERALD_AUTH_MODE</code> must be set ('hmac' or 'jwks').
HERALD_AUTH_HMAC_SECRET	hmac	yes (hmac)	Raw HS256 secret bytes. Empty → <code>NewVerifier</code> errors HMAC mode requires <code>HMACSecret</code> .
HERALD_AUTH_JWKS_URL	jwks	yes (jwks)	HTTPS JWKS endpoint. Empty → <code>NewVerifier</code> errors JWKS mode requires <code>JWKSURL</code> .
HERALD_AUTH_JWKS_TTL	jwks	no	Redis cache TTL for fetched keys. Accepts a Go duration ("5m") or bare seconds ("300"); an unparseable value errors. Unset/zero → defaults to 5 minutes.

Redis (the JWKS key cache) is resolved separately by each flavor's `buildRedis()`; if Redis is absent, JWKS mode still works via the in-memory-only fallback (§4), and HMAC mode never needs it.

§7.1 HMAC quickstart

```
export HERALD_AUTH_MODE=hmac
export HERALD_AUTH_HMAC_SECRET='a-long-random-shared-secret'
cherald serve --http-port 70010 # every /v1/* route now
                                requires a valid HS256 bearer token
```

Callers send `Authorization: Bearer <jwt>` where the JWT is HS256-signed with the same secret and carries at least the `tenant` and `sub` claims (both non-empty).

§7.2 JWKS quickstart

```
export HERALD_AUTH_MODE=jwks
export HERALD_AUTH_JWKS_URL='https://idp.example.com/.well-known/
                                jwks.json'
export HERALD_AUTH_JWKS_TTL=5m # optional; default 5m
# (Redis optional — wired via the flavor's standard Redis env if
# you want cross-restart key caching)
sherald serve --http-port 70011
```

Callers send `Authorization: Bearer <jwt>` signed by a private key whose public key is published in the JWKS (matched by `kid`), using an RS256/RS384/RS512 or ES256/ES384/ES512 algorithm.

§8. Troubleshooting 401s

The middleware only ever returns one of two 401 bodies. Use them to localise the cause.

Response body	Cause	Fix
<code>{"error": "missing bearer token"}</code>	No <code>Authorization</code> header, header missing the <code>Bearer</code> prefix (note the trailing space + case-sensitivity), or the token after <code>Bearer</code> is empty.	Send <code>Authorization: Bearer <jwt></code> exactly.
<code>{"error": "invalid token"}</code>	Verify failed. See the sub-causes below.	—

Sub-causes of `invalid token` (all surface as the same generic 401 to the client — the detail is internal):

- **Signature mismatch.** HMAC: token signed with a different secret than `HERALD_AUTH_HMAC_SECRET`. JWKS: token's `kid` not present in the JWKS (after the forced re-fetch), or the signature does not verify against the published public key.
- **Wrong algorithm.** HMAC mode accepts **only** HS256; JWKS mode accepts **only** the RS/ES family. An `alg: none` token, or HMAC-in-JWKS-mode, is rejected as unexpected signing method.
- **Missing/empty** `kid` (JWKS mode) — token missing `kid`.
- **Expired token** — validated against the verifier's clock (`exp`).
- **Missing/empty required claim** — by default `tenant` or `sub` absent, or present-but-empty-string → token missing `claim/token claim ...` is empty.

Operator-side startup failures (the flavor `serve` won't even start) rather than per-request 401s:

- `HERALD_AUTH_MODE` must be set — `HERALD_AUTH_MODE` unset.
- HMAC mode requires `HMACSecret` — `hmac` mode with an empty secret.
- JWKS mode requires `JWKSURL` — `jwtks` mode with an empty URL.
- `HERALD_AUTH_JWKS_TTL` invalid duration — TTL value is neither a Go duration nor an integer second count.
- JWKS HTTP `<code> / fetch` errors — the JWKS endpoint is unreachable or returned non-200 (JWKS mode, on first key fetch). Redis being down is **not** fatal — keys are loaded into memory regardless.

§9. Testing notes

Tests live in `commons_auth/verifier_test.go` and run with no external services (the JWKS test stands up an `httptest` server; tokens are minted in-process; the clock is injected via `clockwork`):

```
go test -race -count=1 ./commons_auth/...
```

Test	Proves
<code>TestHMACVerifier_VerifiesValidToken</code>	A correctly HS256-signed token with

Test

TestHMACVerifier_RejectsWrongSecret

TestHMACVerifier_RejectsMissingRequiredClaim

TestHMACVerifier_RejectsExpiredToken

TestJWKSVerifier_VerifiesTokenSignedByPubInJWKS

TestGinMiddleware_EndToEnd

TestTenantAndSubjectFromClaims

Proves

required claims verifies and returns the claims.

A token signed with a different secret is rejected.

A token missing a required claim is rejected.

An expired token is rejected (clock-driven).

A token signed by a private key whose public key is published in the JWKS verifies (RS/ES path, real `httptest` JWKS server).

The Gin middleware gates a route: a valid bearer passes and claims land in context; a bad/missing bearer returns 401 — the real end-user-visible behaviour.

`TenantFromClaims/SubjectFromClaims` extract and validate the tenant (UUID) and sub (string) claims, including the error cases.

Anti-bluff observations worth preserving when editing tests:

- The clock is injected (`clockwork`) so the expiry test exercises real exp validation deterministically rather than sleeping.
- `TestGinMiddleware_EndToEnd` drives an actual Gin engine + HTTP round-trip and asserts both the 200-with-claims and the 401 paths — it proves the user-visible gate, not just the verifier in isolation (§107 / Helix §11.4).
- The JWKS test publishes a real generated public key over an `httptest` server, so the `fetch` → `parse` → `verify` pipeline is exercised end-to-end, not mocked.

§10. References

- Source: `commons_auth/verifier.go` (interface + `Config` + constructors), `commons_auth/hmac.go` (HS256 + `requireClaims`), `commons_auth/jwks.go` (RS/ES + JWKS fetch/cache/parse), `commons_auth/middleware.go` (`GinMiddleware` + `ContextKeyClaims`), `commons_auth/claims.go` (`TenantFromClaims/SubjectFromClaims`), `commons_auth/verifier_test.go` (tests).
- Wiring: `commons/cli/serve.go` (`ServeOpts.Middleware` + `RunServe's r.Use` loop); `cherald/cmd/cherald/main.go`, `sherald/cmd/sherald/main.go`, `iherald/cmd/iherald/main.go` (`NewVerifierFromEnv` + `GinMiddleware` threading).

- Catalogue-check: docs/catalogue-checks/HRD-099-commons-auth.md (the §11.4.74 no-match verdict; source comments still cite the executing-time anchor HRD-093).
- Credentials / env setup: docs/guides/OPERATOR_CREDENTIALS.md (env-var resolution order + per-channel/dispatcher setup).
- Spec: docs/specs/mvp/specification.V4.md (flavor-binary + serving-flavor model).
- Dependencies (commons_auth/go.mod): github.com/golang-jwt/jwt/v5 v5.2.1, github.com/redis/go-redis/v9 v9.6.1, github.com/gin-gonic/gin v1.12.0, github.com/google/uuid v1.6.0, github.com/jonboulle/clockwork v0.4.0.

Sources verified

This guide documents internal Herald source only; no external service/library online documentation was relied on. All behavioural claims are grounded in the cited commons_auth/*.go source files (and the commons/cli/serve.go + flavor main.go wiring) as of 2026-05-31.

Verified 2026-05-31: internal doc — no external online sources. Behavioural claims derive from commons_auth/verifier.go, hmac.go, jwks.go, middleware.go, claims.go, verifier_test.go, plus the cli.ServeOpts.Middleware wiring in commons/cli/serve.go and cherald/sherald/iherald main.go (all read 2026-05-31). The dependency versions are the ones pinned in commons_auth/go.mod. Re-verify on a golang-jwt/jwt major-version bump or a commons_auth API change.